UNIPAM Educação que transforma	**CENTRO UNIVERSITÁRIO DE PATOS DE MINAS – UNIPAM** **BACHARELADO EM SISTEMAS DE INFORMAÇÃO** **TURMA: 7º PERÍODO** **DESENVOLVIMENTO DE SISTEMAS DE INFORMAÇÃO AVANÇADOS II** **PROFESSOR RAFAEL MARINHO E SILVA**
	SCROLL EM APLICAÇÕES MOBILE

COMO EXECUTAR OS CÓDIGOS NO CODESPACE DO GITHUB

- Vá para o GitHub e crie um novo repositório.
  - Dê um nome ao repositório, por exemplo,"***react_native***".
  - Abra o repositório criado, e clique no botão Code e selecione *Codespaces*.
  - Crie um novo Codespace ou abra um já criado.
- Para usar os recursos de Scroll, crie um novo projeto React Native executando no terminal, os seguintes comandos:
  - Instale as dependências principais no Codespace com o seguinte comando no terminal para garantir que o ambiente tenha Node.js e npm instalados:

```
sudo apt update
sudo apt install -y nodejs npm
npm install --global expo-cli
```

  - Para criar um novo projeto Expo, siga os comandos descritos em cada exemplo.

NAVEGAÇÃO EM APLICAÇÕES MOBILE

Neste documento, serão abordados quatro projetos que utilizam o conceito de Scroll no React Native, com integração ao Expo e execução via QR Code no Expo Go. O recurso de scroll é fundamental no desenvolvimento mobile, pois permite que interfaces exibam conteúdos que ultrapassam os limites da tela, proporcionando uma navegação contínua e fluida para o usuário. Com o uso de diferentes componentes — como o ScrollView, o FlatList, o SectionList e a integração do KeyboardAvoidingView — os desenvolvedores podem adaptar o layout do aplicativo a diversas necessidades, seja para renderizar listas simples, grandes volumes de dados, agrupamentos categorizados ou formulários interativos que se ajustam à presença do teclado. O objetivo desse conteúdo é explorar e estudar cada um desses conceitos e como optar por uma solução em detrimento da outra, garantindo que a experiência do usuário seja sempre otimizada.

1. SCROLLVIEW COM ITENS ESTILIZADOS

O ScrollView é um componente que permite a rolagem de conteúdos que excedem o tamanho da tela. No projeto com itens estilizados, o ScrollView é utilizado para exibir uma lista de elementos criados dinamicamente. Cada item é renderizado com um conjunto de estilos aplicados, permitindo personalização visual e espaçamento uniforme entre os elementos.

- **Características:**
  - **Simplicidade:** Ideal para conteúdos de tamanho limitado ou quando a quantidade de itens é pequena a moderada.
  - **Estilização Flexível:** Permite aplicar estilos tanto no contêiner principal quanto nos itens individuais, possibilitando uma apresentação visual customizada.
  - **Layout Linear:** Os elementos são renderizados em sequência, normalmente de forma vertical, o que facilita a leitura e a organização do conteúdo.
- **Vantagens:**
  - Fácil implementação e entendimento, pois renderiza todos os elementos de uma só vez.
  - Bom para listas curtas ou interfaces onde o volume de dados não impacta a performance.
- **Desvantagens:**
  - Não é otimizado para grandes volumes de dados, pois renderiza todos os itens simultaneamente, o que pode causar problemas de performance e consumo excessivo de memória.
- **Cenários de Uso:**
  - Utilize o ScrollView com itens estilizados quando o número de elementos for limitado e a ênfase for na personalização e na apresentação visual dos itens, sem necessidade de otimizações para renderização massiva.
- Comandos para criar projetos com ScrollView e com itens estilizados:

```
npx create-expo-app ex01 --template blank
```

```
cd ex01
```

- Substitua o conteúdo de App.js por:

```
import React from "react";
import { ScrollView, Text, StyleSheet, View } from "react-native";

export default function App(){
  return (
    <ScrollView contentContainerStyle={styles.container}>
      {Array.from({ length: 20}).map((_, index) => (
        <View key={index} style={styles.item}>
          <Text style={styles.text}>Item {index + 1}</Text>
        </View>
      ))}
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: {
    padding: 20,
    marginTop: 20,
  },
  item: {
    marginBottom: 10,
    padding: 20,
```

```
    backgroundColor: '#f0f0f0',
    borderRadius: 8,
  },
  text: {
    fontSize: 16,
    fontWeight: 'bold',
  },
})
```

- Execute o projeto Expo:

```
npx expo start --tunnel
```

2. FLATLIST COM GRANDE VOLUME DE DADOS

O FlatList é um componente especializado para renderização de listas longas e dinâmicas. Ele implementa a técnica de "virtualização", renderizando apenas os itens visíveis na tela e carregando os demais conforme o usuário rola a lista. Isso torna o gerenciamento de listas extensas muito mais eficiente em termos de performance.

- **Características:**
  - **Virtualização:** Renderiza somente os itens que estão na viewport, economizando recursos do dispositivo.
  - **Chave Única:** Utiliza o método de chave (keyExtractor) para identificar cada item de forma única, facilitando atualizações e manipulações na lista.
  - **Renderização Condicional:** Possibilita a definição de funções para renderizar cada item (renderItem), permitindo maior controle sobre a apresentação dos dados.
- **Vantagens:**
  - Alta performance e escalabilidade, ideal para grandes volumes de dados.
  - Redução do consumo de memória e melhoria na responsividade do app.
- **Desvantagens:**
  - Requer um pouco mais de configuração e entendimento dos conceitos de virtualização.
  - Pode não ser tão flexível quanto o ScrollView para casos onde o layout precisa ser altamente customizado ou quando a quantidade de dados é pequena.
- **Cenários de Uso:**
  - Utilize o FlatList quando se espera um grande volume de dados a ser exibido, como listas de produtos, mensagens ou qualquer outro conteúdo dinâmico que possa crescer com o tempo. Ele é perfeito para cenários que demandam performance e escalabilidade.
- Comandos para criar projetos com FlatList:

```
npx create-expo-app ex02 --template blank
```

```
cd ex02
```

- Substitua o conteúdo de App.js por:

```
import React from "react";
import { FlatList, Text, StyleSheet, View } from 'react-native';

const data = Array.from({ length: 80}, (_, i) => ({ id: i, name: `Item ${i + 1}`}));

export default function App(){
  return(
    <FlatList
      data={data}
      keyExtractor={(item) => item.id.toString()}
      renderItem={({ item }) => (
        <View style={styles.item}>
          <Text style={styles.text}>{item.name}</Text>
        </View>
      )}
      contentContainerStyle={styles.container}
    />
  );
}

const styles = StyleSheet.create({
  container: {
    padding: 20,
    marginTop: 20,
  },
  item: {
    marginBottom: 10,
    padding: 20,
    backgroundColor: '#e0f7fa',
    borderRadius: 8,
  },
  text: {
    fontSize: 16,
    fontWeight: 'bold',
  },
});
```

- Execute o projeto Expo:

```
npx expo start --tunnel
```

3. SECTIONLIST COM CABEÇALHOS

O SectionList é uma extensão do FlatList que permite agrupar itens em seções, cada uma com um cabeçalho descritivo. Essa estrutura facilita a organização de dados quando há uma categorização lógica, como agrupamento por datas, categorias ou temas.

- **Características:**
  - **Agrupamento:** Permite definir seções (groups) e associar a cada uma um cabeçalho, melhorando a navegação e a compreensão do conteúdo.
  - **Renderização de Cabeçalhos:** Possibilita a customização dos cabeçalhos de seção, que podem ter estilos e layouts diferenciados dos itens.
  - **Virtualização com Seções:** Assim como o FlatList, ele também virtualiza os itens, garantindo eficiência mesmo com grandes volumes de dados.

- **Vantagens:**
  - Excelente para organizar dados que possuem uma estrutura hierárquica ou lógica de agrupamento.
  - Melhora a experiência do usuário ao facilitar a identificação e navegação por diferentes categorias dentro da lista.
- **Desvantagens:**
  - Pode ser mais complexo de configurar se comparado a um simples ScrollView, especialmente para desenvolvedores iniciantes.
  - Requer um bom entendimento de como estruturar os dados em seções para que a listagem seja efetiva.
- **Cenários de Uso:**
  - O SectionList deve ser utilizado quando os dados precisam ser agrupados de forma lógica e visual, como listas de contatos organizados por letra inicial, calendários de eventos, ou qualquer situação em que a categorização facilite a navegação e a compreensão do conteúdo pelo usuário.
- Comandos para criar projetos com SectionList:

```
npx create-expo-app ex03 --template blank
```

```
cd ex03
```

- Substitua o conteúdo de App.js por:

```
import React from "react";
import { SectionList, Text, StyleSheet, View, } from 'react-native';

const sections = [
  { title: 'Seção 1', data: [' Item 1', 'Item 2', 'Item 3', 'Item 4', 'Item 4', 'Item 6'] },
  { title: 'Seção 2', data: [' Item 7', 'Item 8', 'Item 9', 'Item 10', 'Item 11'] },
  { title: 'Seção 3', data: [' Item 12', 'Item 13', 'Item 14', 'Item 15'] },
];

export default function App() {
  return (
    <View style={styles.container}>
      <SectionList
        sections={sections}
        keyExtractor={(item, index) => item + index}
        renderItem={({ item }) => (
          <View style={styles.item}>
            <Text style={styles.text}>{item}</Text>
          </View>
        )}
        renderSectionHeader={({ section }) => (
          <Text style={styles.header}>{section.title}</Text>
        )}
        contentContainerStyle={StyleSheet.list}
      />
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    marginTop: 40,
  },
  list: {
    padding: 20,
    marginTop: 80,
  },
  header: {
    fontSize: 18,
    fontWeight: 'bold',
    backgroundColor: '#d0d0d0',
    padding: 10,
    borderRadius: 8,
  },
  item: {
    marginBottom: 10,
    padding: 15,
    backgroundColor: '#f9fbe7',
    borderRadius: 8,
  },
  text: {
    fontSize: 16,
  },
});
```

- Execute o projeto Expo:

```
npx expo start --tunnel
```

4. FORMULÁRIO COM KEYBOARDAVOIDINGVIEW

Em interfaces que envolvem entrada de dados, como formulários, o teclado virtual pode sobrepor os campos de entrada, dificultando a interação do usuário. O KeyboardAvoidingView é uma solução para esse problema, ajustando automaticamente o layout da tela para evitar que o teclado cubra os elementos interativos. Ao ser combinado com um ScrollView, permite que o usuário role a tela para acessar todos os campos, mesmo quando o teclado estiver ativo.

- **Características:**
  - **Ajuste Dinâmico:** Modifica o posicionamento do conteúdo para que os campos de entrada permaneçam visíveis mesmo com o teclado aberto.
  - **Integração com ScrollView:** Permite que a área do formulário seja rolável, garantindo que todos os campos sejam acessíveis.
  - **Configurações de Comportamento:** Diferentes comportamentos podem ser configurados (como “padding” ou “position”) para atender melhor às necessidades do layout.
- **Vantagens:**
  - Melhora significativamente a usabilidade em formulários, prevenindo a obstrução dos campos de entrada pelo teclado.
  - Proporciona uma experiência de usuário mais fluida e confortável, especialmente em dispositivos com telas menores.

- **Desvantagens:**
  - Pode exigir ajustes finos de layout e testes em diferentes dispositivos para garantir que o comportamento seja consistente.
  - Em alguns casos, a combinação de componentes pode gerar desafios na manutenção do design responsivo.
- **Cenários de Uso:**
  - Utilize o KeyboardAvoidingView em qualquer aplicação que envolva formulários ou entradas de dados, especialmente quando esses campos estiverem posicionados na parte inferior da tela. Essa abordagem é essencial para melhorar a acessibilidade e garantir que os usuários possam interagir com todos os elementos do formulário sem dificuldades, independentemente do dispositivo utilizado.
- Comandos para criar projetos com KeyboardAvoidingView:

```
npx create-expo-app ex04 --template blank
```

```
cd ex04
```

- Substitua o conteúdo de App.js por:

```
import React, { useState } from "react";
import {
  ScrollView,
  KeyboardAvoidingView,
  TextInput,
  Text,
  StyleSheet,
  View,
  Button,
} from "react-native";

const FormScreen = () => {
  // Inicializa um array com 10 entradas vazias.
  const [inputs, setInputs] = useState(Array(10).fill(""));
  const handleInputChange = (text, index) => {
    const updatedInputs = [...inputs];
    updatedInputs[index] = text;
    setInputs(updatedInputs);
  };

  return (
    <KeyboardAvoidingView behavior="padding" style={styles.container}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>
        <Text style={styles.title}>Formulário</Text>
        {Array.from({ length: 10 }).map((_, index) => (
          <TextInput
            key={index}
            style={styles.input}
            placeholder={`Campo ${index + 1}`}
            value={inputs[index]} // Valor correspondente ao índice
            // Atualiza o valor do índice específico
            onChangeText={(text) => handleInputChange(text, index)}
          />
```

```
        ))}
        <Button
          title="Enviar"
          onPress={() => alert(`Dados enviados: ${JSON.stringify(inputs)}`)}
        />
      </ScrollView>
    </KeyboardAvoidingView>
  );
};

const styles = StyleSheet.create({
  container: {
    marginTop: 40,
  },
  scrollContainer: {
    padding: 20,
  },
  title: {
    fontSize: 24,
    fontWeight: "bold",
    marginBottom: 20,
  },
  input: {
    borderWidth: 1,
    borderColor: "#ccc",
    padding: 10,
    marginBottom: 10,
    borderRadius: 8,
  },
});

export default FormScreen;
```

- Execute o projeto Expo:

```
npx expo start --tunnel
```